BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. They shift one position right to make space.

Q2. [1,3,4].

Q3. 4.

Q4. The input is already correctly ordered; comparisons find no element greater than the key.

You Do pages · Independent questions

Q1. [1,2,5,6].

Q2. Merge pairs → [2,6] and [1,5]; merge both → [1,2,5,6].

Q3. There are about $\log_2 n$ merge levels, and each level processes n elements → O(n log n) time, with O(n) extra array storage for this implementation.

Exit Ticket · Model answer

False. Both input runs must already be sorted.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

Do Now Recall previous learning | I Do Read new knowledge | We Do Practise with guidance | You Do Work independently | Review & Improve Check understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 186 · Merge sort: trace and implement

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. In insertion sort, what happens to values greater than the key in the sorted prefix?

Q2. After one ascending bubble-sort pass on [4,1,3], what remains?

Q3. What is the length of int[] a={8,5,2,1}?

Q4. Why is swapping not needed to place key 6 into already sorted [2,4,6,8]?

Do Now Recall previous learning | I Do Read new knowledge | We Do Practise with guidance | You Do Work independently | Review & Improve Check understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

run — A consecutive sorted block.

merge — Combine sorted sequences while preserving order.

pointer — Position of the next candidate in a run.

bottom-up — Build sorted runs iteratively from small to large.

O(n log n) — Common time bound for comparison-based merge sort.

New learning

Start with ordered runs — At each merge stage, both input runs must already be sorted.

Compare front values — Append the smaller current value, then advance that run's pointer.

Copy the remainder — When one run finishes, copy all remaining values from the other run; repeated merging doubles run size.

Worked example

Merge the sorted runs [1,8] and [4,5].

1. Predict first — Compare the two current front values at every step.

2. Model solution — Choose 1, then 4, then 5, then copy the remaining 8 → [1,4,5,8].

We Do · Practise with guidance

Merge [2,9] and [3,7] without sorting the runs again.

Predict first

Which value enters the output first?

Model solution

2, then 3, then 7, then 9: [2,3,7,9].

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Merge [2,6] with [1,5].

Question 2

Bottom-up merge sort [6,2,5,1]: show the size-two runs then final result.

Question 3

Explain the O(n log n) time complexity of merge sort.

Exit Ticket · Answer before checking

Two unsorted arrays can be passed to the standard merge step and always yield sorted output.